JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class DataInputStream

java.lang.Object
    java.io.InputStream
        java.io.FilterInputStream
            java.io.DataInputStream

All Implemented Interfaces:

Closeable, DataInput, AutoCloseable

```
public class DataInputStream
extends FilterInputStream
implements DataInput
```

A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way. An application uses a data output stream to write data that can later be read by a data input stream.

DataInputStream is not necessarily safe for multithreaded access. Thread safety is optional and is the responsibility of users of methods in this class.

Since:

JDK1.0

See Also:

DataOutputStream

Field Summary

Fields inherited from class java.io.FilterInputStream

in

Constructor Summary

Constructors

Constructor	Description
`DataInputStream(InputStream in)`	Creates a DataInputStream that uses the specified underlying InputStream.

Method Summary

Deprecat

Modifier and Type	Method	Description
`int`	`read(byte[] b)`	Reads some number of bytes from the contained into the buffer array `b`.
`int`	`read(byte[] b, int off, int len)`	Reads up to `len` bytes of data from the containe bytes.
`boolean`	`readBoolean()`	See the general contract of the `readBoolean` me
`byte`	`readByte()`	See the general contract of the `readByte` metho
`char`	`readChar()`	See the general contract of the `readChar` metho
`double`	`readDouble()`	See the general contract of the `readDouble` met
`float`	`readFloat()`	See the general contract of the `readFloat` meth
`void`	`readFully(byte[] b)`	See the general contract of the `readFully` meth
`void`	`readFully(byte[] b, int off, int len)`	See the general contract of the `readFully` meth
`int`	`readInt()`	See the general contract of the `readInt` method
`String`	`readLine()`	**Deprecated.** This method does not properly convert bytes to preferred way to read lines of text is via the `Buf` method. Programs that use the `DataInputStrea` converted to use the `BufferedReader` class by re `DataInputStream d = new DataInp` with:

		BufferedReader d = new BufferedReader(new I
long	**readLong**()	See the general contract of the readLong metho
short	**readShort**()	See the general contract of the readShort meth
int	**readUnsignedByte**()	See the general contract of the readUnsignedBy
int	**readUnsignedShort**()	See the general contract of the readUnsignedSh
String	**readUTF**()	See the general contract of the readUTF method
static **String**	**readUTF**(**DataInput** in)	Reads from the stream in a representation of a encoded in **modified UTF-8** format; this string a String.
int	**skipBytes**(int n)	See the general contract of the skipBytes meth

Methods inherited from class java.io.FilterInputStream

available, close, mark, markSupported, read, reset, skip

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

DataInputStream

```
public DataInputStream(InputStream in)
```

Creates a DataInputStream that uses the specified underlying InputStream.

Parameters:

in - the specified input stream

Method Detail

read

```
public final int read(byte[] b)
               throws IOException
```

Reads some number of bytes from the contained input stream and stores them into the buffer array `b`. The number of bytes actually read is returned as an integer. This method blocks until input data is available, end of file is detected, or an exception is thrown.

If `b` is null, a `NullPointerException` is thrown. If the length of `b` is zero, then no bytes are read and `0` is returned; otherwise, there is an attempt to read at least one byte. If no byte is available because the stream is at end of file, the value `-1` is returned; otherwise, at least one byte is read and stored into `b`.

The first byte read is stored into element `b[0]`, the next one into `b[1]`, and so on. The number of bytes read is, at most, equal to the length of `b`. Let `k` be the number of bytes actually read; these bytes will be stored in elements `b[0]` through `b[k-1]`, leaving elements `b[k]` through `b[b.length-1]` unaffected.

The `read(b)` method has the same effect as:

```
read(b, 0, b.length)
```

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b` - the buffer into which the data is read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`IOException` - if the first byte cannot be read for any reason other than end of file, the stream has been closed and the underlying input stream does not support reading after close, or another I/O error occurs.

See Also:

`FilterInputStream.in`, `InputStream.read(byte[], int, int)`

read

```
public final int read(byte[] b,
                      int off,
                      int len)
               throws IOException
```

Reads up to `len` bytes of data from the contained input stream into an array of bytes. An attempt is made to read as many as `len` bytes, but a smaller number may be read, possibly zero. The number of bytes actually read is returned as an integer.

This method blocks until input data is available, end of file is detected, or an exception is thrown.

If `len` is zero, then no bytes are read and `0` is returned; otherwise, there is an attempt to read at least one byte. If no byte is available because the stream is at end of file, the value `-1` is returned; otherwise, at least one byte is read and stored into `b`.

The first byte read is stored into element `b[off]`, the next one into `b[off+1]`, and so on. The number of bytes read is, at most, equal to `len`. Let *k* be the number of bytes actually read; these bytes will be stored in elements `b[off]` through `b[off+`*k*`-1]`, leaving elements `b[off+`*k*`]` through `b[off+len-1]` unaffected.

In every case, elements `b[0]` through `b[off]` and elements `b[off+len]` through `b[b.length-1]` are unaffected.

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b` - the buffer into which the data is read.

`off` - the start offset in the destination array `b`

`len` - the maximum number of bytes read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`NullPointerException` - If `b` is null.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

`IOException` - if the first byte cannot be read for any reason other than end of file, the stream has been closed and the underlying input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in, InputStream.read(byte[], int, int)

readFully

```
public final void readFully(byte[] b)
                     throws IOException
```

See the general contract of the readFully method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readFully in interface DataInput

Parameters:

b - the buffer into which the data is read.

Throws:

EOFException - if this input stream reaches the end before reading all the bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readFully

```
public final void readFully(byte[] b,
                            int off,
                            int len)
                     throws IOException
```

See the general contract of the readFully method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readFully in interface DataInput

Parameters:

b - the buffer into which the data is read.

off - the start offset of the data.

len - the number of bytes to read.

Throws:

EOFException - if this input stream reaches the end before reading all the bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

skipBytes

```
public final int skipBytes(int n)
                    throws IOException
```

See the general contract of the skipBytes method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

skipBytes in interface DataInput

Parameters:

n - the number of bytes to be skipped.

Returns:

the actual number of bytes skipped.

Throws:

IOException - if the contained input stream does not support seek, or the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

readBoolean

```
public final boolean readBoolean()
                           throws IOException
```

See the general contract of the readBoolean method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readBoolean in interface DataInput

Returns:

the boolean value read.

Throws:

EOFException - if this input stream has reached the end.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readByte

```
public final byte readByte()
                    throws IOException
```

See the general contract of the readByte method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readByte in interface DataInput

Returns:

the next byte of this input stream as a signed 8-bit byte.

Throws:

EOFException - if this input stream has reached the end.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readUnsignedByte

```
public final int readUnsignedByte()
                           throws IOException
```

See the general contract of the readUnsignedByte method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readUnsignedByte in interface DataInput

Returns:

the next byte of this input stream, interpreted as an unsigned 8-bit number.

Throws:

EOFException - if this input stream has reached the end.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readShort

```
public final short readShort()
                       throws IOException
```

See the general contract of the readShort method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readShort in interface DataInput

Returns:

the next two bytes of this input stream, interpreted as a signed 16-bit number.

Throws:

EOFException - if this input stream reaches the end before reading two bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readUnsignedShort

```
public final int readUnsignedShort()
                              throws IOException
```

See the general contract of the readUnsignedShort method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readUnsignedShort in interface DataInput

Returns:

the next two bytes of this input stream, interpreted as an unsigned 16-bit integer.

Throws:

EOFException - if this input stream reaches the end before reading two bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readChar

```
public final char readChar()
                    throws IOException
```

See the general contract of the readChar method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readChar in interface DataInput

Returns:

the next two bytes of this input stream, interpreted as a char.

Throws:

EOFException - if this input stream reaches the end before reading two bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readInt

```
public final int readInt()
                  throws IOException
```

See the general contract of the readInt method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readInt in interface DataInput

Returns:

the next four bytes of this input stream, interpreted as an int.

Throws:

EOFException - if this input stream reaches the end before reading four bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readLong

```
public final long readLong()
                    throws IOException
```

See the general contract of the readLong method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readLong in interface DataInput

Returns:

the next eight bytes of this input stream, interpreted as a long.

Throws:

EOFException - if this input stream reaches the end before reading eight bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

FilterInputStream.in

readFloat

```
public final float readFloat()
                     throws IOException
```

See the general contract of the readFloat method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readFloat in interface DataInput

Returns:

the next four bytes of this input stream, interpreted as a float.

Throws:

EOFException - if this input stream reaches the end before reading four bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

readInt(), Float.intBitsToFloat(int)

readDouble

```
public final double readDouble()
                        throws IOException
```

See the general contract of the readDouble method of DataInput.

Bytes for this operation are read from the contained input stream.

Specified by:

readDouble in interface DataInput

Returns:

the next eight bytes of this input stream, interpreted as a double.

Throws:

EOFException - if this input stream reaches the end before reading eight bytes.

IOException - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

See Also:

readLong(), Double.longBitsToDouble(long)

readLine

```
@Deprecated
public final String readLine()
                                    throws IOException
```

Deprecated. *This method does not properly convert bytes to characters. As of JDK 1.1, the preferred way to read lines of text is via the BufferedReader.readLine() method. Programs that use the*

DataInputStream class to read lines can be converted to use the BufferedReader class by replacing code of the form:

```
DataInputStream d = new DataInputStream(in);
```

with:

```
BufferedReader d
     = new BufferedReader(new InputStreamReader(in));
```

See the general contract of the `readLine` method of `DataInput`.

Bytes for this operation are read from the contained input stream.

Specified by:

`readLine` in interface `DataInput`

Returns:

the next line of text from this input stream.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`BufferedReader.readLine()`, `FilterInputStream.in`

readUTF

```
public final String readUTF()
                     throws IOException
```

See the general contract of the `readUTF` method of `DataInput`.

Bytes for this operation are read from the contained input stream.

Specified by:

`readUTF` in interface `DataInput`

Returns:

a Unicode string.

Throws:

`EOFException` - if this input stream reaches the end before reading all the bytes.

`IOException` - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

`UTFDataFormatException` - if the bytes do not represent a valid modified UTF-8 encoding of a string.

See Also:

readUTF(java.io.DataInput)

readUTF

```
public static final String readUTF(DataInput in)
                            throws IOException
```

Reads from the stream `in` a representation of a Unicode character string encoded in modified UTF-8 format; this string of characters is then returned as a `String`. The details of the modified UTF-8 representation are exactly the same as for the `readUTF` method of `DataInput`.

Parameters:

`in` - a data input stream.

Returns:

a Unicode string.

Throws:

`EOFException` - if the input stream reaches the end before all the bytes.

`IOException` - the stream has been closed and the contained input stream does not support reading after close, or another I/O error occurs.

`UTFDataFormatException` - if the bytes do not represent a valid modified UTF-8 encoding of a Unicode string.

See Also:

readUnsignedShort()

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.